Experiment No. 6
Apply Random Forest algorithm for credit fraud detection.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Apply Random Forest algorithm for credit fraud detection.

Objective:

• To understand the concept of Ensemble Learning.
• To implement Random Forest algorithm for classification.
• To detect fraudulent transactions using machine learning.
• To analyze model performance on real-world data.

Theory:

Credit Fraud Detection:

Credit fraud detection involves identifying fraudulent transactions from a large set of financial data. It is a classification problem where transactions are labeled as fraudulent or non-fraudulent.

Random Forest Algorithm:

Random Forest is an ensemble learning technique that builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting.

Key Concepts:

Decision Trees: Base models used in Random Forest

Bagging (Bootstrap Aggregation): Random sampling of data

Feature Randomness: Random selection of features for each tree

Voting Mechanism: Final output based on majority voting

Advantages:

- High accuracy
- Handles large datasets
- Reduces overfitting
- Works well with missing data

Implementation:

• Import required libraries such as NumPy, Pandas, and Scikit-learn.

• Load the credit card transaction dataset.

• Preprocess data (handle missing values, normalize features).

• Split dataset into training and testing sets.

• Initialize Random Forest classifier.

• Train the model using training data.

• Predict results on test data.

• Evaluate model performance using accuracy, precision, and recall.

Code:

```
# Install dependencies:

# pip install pandas numpy scikit-learn matplotlib seaborn


import pandas as pd

import numpy as np

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, precision_score, recall_score, classification_report


# Step 1: Load dataset

# Replace with your dataset path (e.g., creditcard.csv from Kaggle)

data = pd.read_csv("creditcard.csv")
```

```
# Step 2: Explore dataset

print(data.head())

print(data['Class'].value_counts())  # 0 = Normal, 1 = Fraud


# Step 3: Handle imbalance (optional - sampling)

fraud = data[data['Class'] == 1]

normal = data[data['Class'] == 0].sample(len(fraud))


data_balanced = pd.concat([fraud, normal])


# Step 4: Split features and labels

X = data_balanced.drop('Class', axis=1)

y = data_balanced['Class']


# Step 5: Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Step 6: Initialize Random Forest

model = RandomForestClassifier(n_estimators=100, random_state=42)


# Step 7: Train model

model.fit(X_train, y_train)
```

```
# Step 8: Predictions

y_pred = model.predict(X_test)


# Step 9: Evaluation

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Precision:", precision_score(y_test, y_pred))

print("Recall:", recall_score(y_test, y_pred))


print("\nClassification Report:\n")

print(classification_report(y_test, y_pred))
```

Conclusion:

Random Forest is an effective algorithm for fraud detection. It improves prediction accuracy by combining multiple decision trees and helps in identifying fraudulent transactions efficiently.